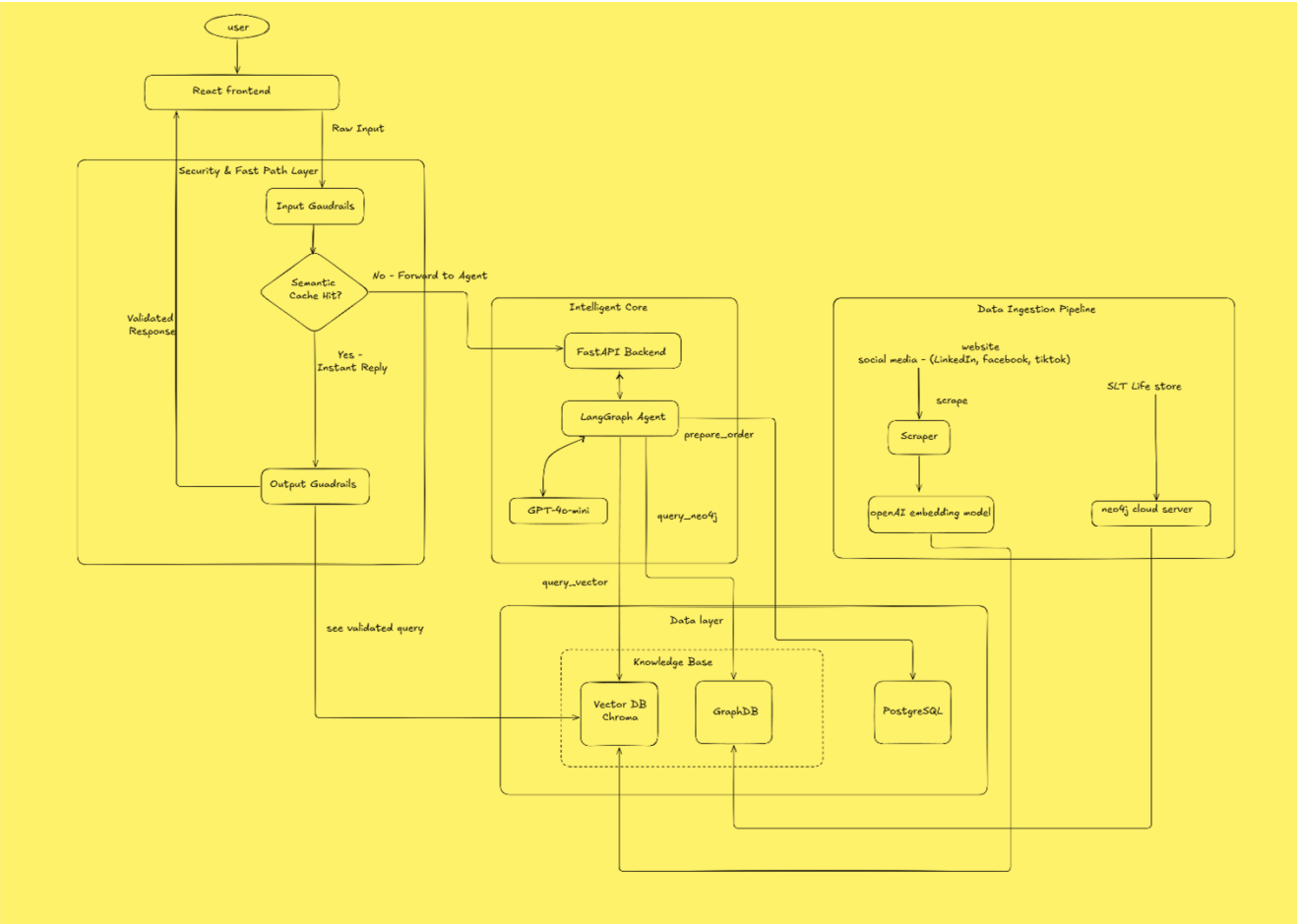
AI Enterprise Agent

1. Architecture Overview: AI Enterprise Agent

The AI Enterprise Agent is a sophisticated e-commerce ecosystem that is used to bridge the gap between structured data (Neo4j, PostgreSQL) and unstructured web content (Chroma DB). To ensure enterprise-grade speed and safety, the architecture now incorporates a high-speed Semantic Cache and strict AI Guardrails.



System Architecture Flow (Updated)

1. **User Input** → React Frontend
2. **Security Layer** → Input Guardrails (Safety & Relevance check)
3. **Fast Path** → Semantic Cache (ChromaDB) → If Hit, return instant response.
4. **Intelligent Core (If Cache Miss)** → LangGraph Agent (GPT-4o-mini)
5. **Background Processes** → Scraper Runner → Data Layer
6. **Data Layer** → Neo4j (Product Graph), ChromaDB (Vector Search & Cache), PostgreSQL (Operations)

Core Technology Stack

- **Orchestration:** LangGraph (State Machine Agent).
- **API Layer:** FastAPI (Asynchronous Python).
- **Frontend:** React 18 (Vite, Tailwind CSS).
- **Graph DB (Neo4j):** Product hierarchies, specs, and relationships.
- **Vector DB (Chroma):** Semantic embeddings for RAG (LinkedIn, TikTok, Web content), **and Semantic Response Caching**.
- **Relational DB (PostgreSQL):** ACID-compliant storage for Users, Inventory, and Orders.
- **Safety & Compliance:** Custom Guardrails for input sanitization and output verification.

2. Backend & Agent Logic

The backend centers on a LangGraph state machine that manages the reasoning cycle. The workflow has been optimized for reduced latency and strict brand safety.

LangGraph Workflow

- **Input Guardrails (New):** Evaluates the user's raw input to block malicious prompts, off-topic queries, or unsafe requests before processing begins.
- **Rewrite:** Formulates a standalone search query based on chat history.
- **Semantic Cache Check (New):** Compares the standalone query against a ChromaDB cache using a strict similarity threshold (e.g., 0.85). If a semantic match is found, the system bypasses the LLM and instantly returns the cached response.
- **Intent Router:** Classifies intent into four categories: graph_db, vector_db, order_action, or general_chat.
- **Tool Execution:**
    - query_neo4j: Retrieves precise technical specs.
    - query_vector: Fetches company context/social sentiment.
    - prepare_order: Verifies inventory in PostgreSQL and triggers the UI form signal.

- **Synthesis:** Combines data into a natural language response.
- **Output Guardrails (New):** Verifies the synthesized response to ensure no hallucinations, inappropriate language, or unverified claims are presented to the user.
- **Cache Update (New):** Saves the successfully validated standalone query and its corresponding final answer to the Semantic Cache for future users.

Key API Endpoints

- POST /v1/chat: Main agent interaction.
- POST /auth/login: Session management.
- GET /admin/orders: Relational data retrieval for admins.

3. Frontend & Intelligent UI

The React-based interface is designed for "Contextual Commerce," where the UI reacts to the Agent's reasoning.

Dynamic Components

- **Message Interception:** The UI scans for signals like [SHOW_ORDER_FORM] or [REQUIRE_LOGIN] to render interactive components directly in the chat thread.
- **Product Canvas:** A side-panel that visualizes product specs and live pricing automatically when a product is mentioned.
- **Admin Dashboard:** A separate route for CRUD operations on products and order status management (Pending → Shipped).

4. Database Schema

The system uses a Tri-Database Architecture to manage different data types efficiently: Relational (Business), Graph (Product Intelligence), and Vector (Contextual Knowledge).

1. PostgreSQL (Business & Transactions)

Handles structured data requiring ACID compliance.

- customers: Stores user profiles, authentication (hashed passwords), and contact info.
- products: The master inventory list. Includes SKU, name, base_price, and stock_level.
- orders: Transaction records linking customers to their purchases. Includes status (Pending, Shipped, Delivered).

- order_items: Junction table for line items, quantities, and historical price snapshots.

2. Neo4j (Product Graph)

Maps complex relationships and technical hierarchies.

- **Nodes:**
    - (: Product): Individual items (e.g., X500 Router).
    - (: Category): Product groupings (e.g., Networking, Fiber).
    - (: Specification): Technical attributes (e.g., 5GHz, WiFi-6).
- **Relationships:**
    - [: BELONGS_TO]: Links a Product to its Category.
    - [: HAS_SPEC]: Links a Product to its technical features.
    - [: COMPATIBLE_WITH]: Defines cross-selling or technical compatibility between items.

3. Chroma DB (Knowledge & Cache Vector Store)

Stores unstructured "context" for RAG (Retrieval-Augmented Generation) and manages the Semantic Cache.

- **Collection 1:** slt_knowledge_base
    - **Data Points:** Fragments of text from Website URLs, LinkedIn posts, Facebook updates, and TikTok transcripts.
    - **Metadata:** Each vector includes source_url, platform (e.g., "LinkedIn"), and timestamp.
- **Collection 2 (New):** semantic_response_cache
    - **Data Points:** Standalone user queries and their validated AI-generated responses.
    - **Metadata:** Timestamps and performance metrics to track cache hit rates.

5. Ingestion Pipeline & Data Sync Flow

A unified pipeline (main_scraper.py) that synchronizes all databases.

- **Extraction:** Scrapes Website, LinkedIn, and TikTok.
- **Structural Mapping:** Converts product tables into Cypher queries for Neo4j.

- **Vectorization:** Fragments text and generates embeddings for Chroma DB.
- **Inventory Sync:** Populates PostgreSQL with master SKU and stock data.
- **Cache Invalidation (New):** Performs a physical reset of the ChromaDB Semantic Cache folder during data ingestion or admin updates. This ensures users never receive stale data or outdated pricing from the cache.